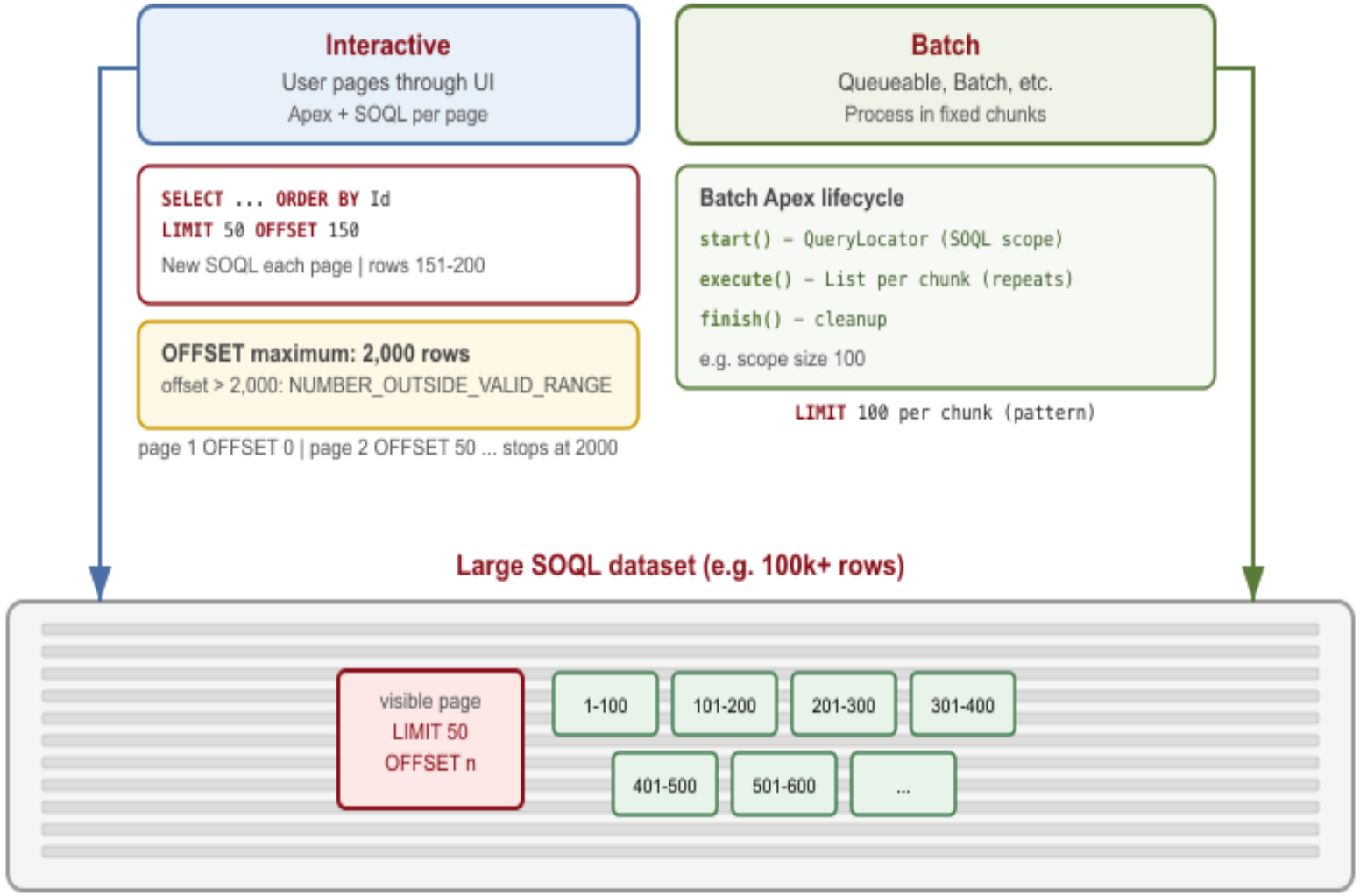
Scaling Apex and LWC with Apex Cursors

#LDNsCall #LC26

Andrew Fawcett, AndyInTheCloud Consulting Ltd
andy@andyinthecloud.com
https://www.linkedin.com/in/andyfawcett/

Accessing and processing datasets today in SOQL

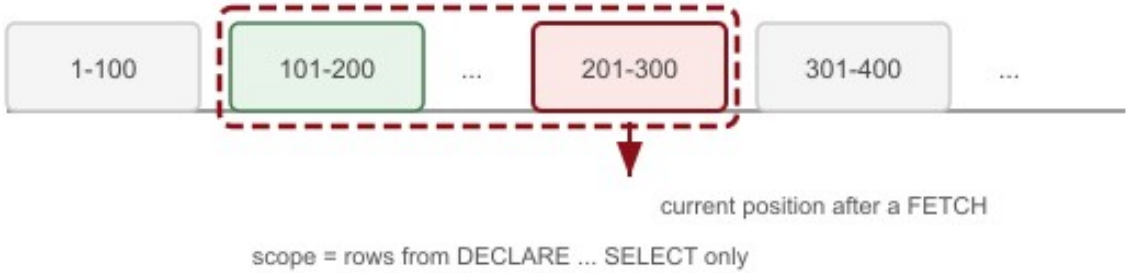
UI paging with OFFSET/LIMIT vs async processing in fixed-size chunks.



What is a SQL Cursor?

Traverse a query result incrementally instead of returning the full set at once.

1. One declared result set - position moves along rows



Sections 2 and 3 = two views of the same idea

2 = lifecycle statements (DECLARE ... CLOSE)

3 = access patterns (how FETCH moves in that scope)

Different from OFFSET paging (new query per page)

2. Lifecycle pattern (SQL statements)

Ordered steps the application executes

1. `DECLARE c CURSOR FOR <select>;`
   Defines query scope
2. `OPEN c;`
   Before first row
3. `FETCH ... FROM c INTO ...;` (repeat)
   NEXT, PRIOR, ABSOLUTE...
4. `CLOSE c;`
   Release resources
5. `DEALLOCATE c;` (some products)
   Drop definition

Example

```
DECLARE account_cur CURSOR FOR
  SELECT id, name FROM accounts
  WHERE active ORDER BY id;
OPEN account_cur;
FETCH NEXT FROM account_cur ...
CLOSE account_cur;
```

3. Access patterns (how you read rows)

FETCH orientations - same open cursor, move position in scope

Forward-only cursor (typical default)

`FETCH NEXT` – one row forward
Most common in ETL / streaming loops

Scrollable cursor (when supported)

`FETCH NEXT | PRIOR | FIRST | LAST`

`FETCH ABSOLUTE n | FETCH RELATIVE n`

Jump relative to start or current position (product-dependent)

Not a cursor access pattern:

`SELECT ... OFFSET 20 FETCH 10`

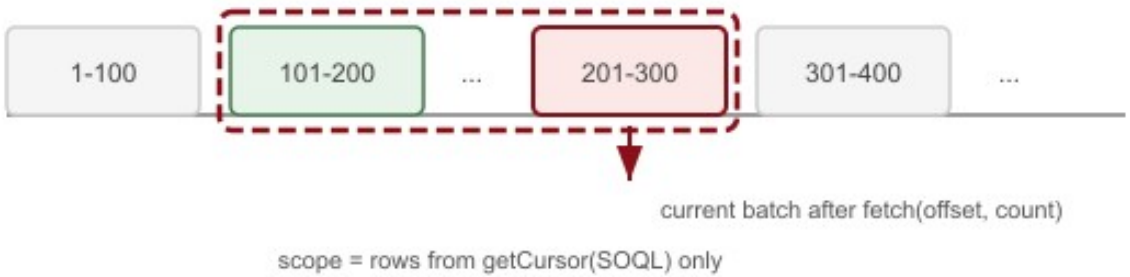
new query each page - no held position

cursor: one OPEN, many FETCH in same scope

What is a SOQL Cursor?

SQL cursor concepts on Salesforce: Database.getCursor(SOQL) and incremental fetch in Apex.

1. One SOQL result set - position moves along rows



Apex maps SQL cursor ideas to API calls

2 = lifecycle (getCursor, fetch, hold state)

3 = access patterns (offset batch vs fetchPage)

Not [SELECT ...] loading all rows at once

2. Lifecycle pattern (Apex API)

No DECLARE/OPEN/CLOSE - platform cursor over SOQL

1. `Database.getCursor(soql);`
   Or PaginationCursor
2. `cursor.getNumRecords();`
   Optional row count
3. `cursor.fetch(offset, count);`
   Repeat until done
4. Hold cursor: LWC or Cache
   Available across requests
5. Governed by Limits.*ApexCursor*
   Apex cursor limits

Example (this repo)

```
Database.Cursor c =
  Database.getCursor(
    'SELECT Id, Name FROM Account
    WHERE Name LIKE 'TEST%'
    ORDER BY Name',
  AccessLevel.USER_MODE);
List<Account> batch =
  c.fetch(offset, pageSize);
```

3. Access patterns (how you read rows)

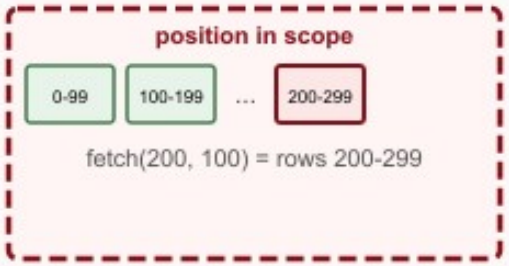
Incremental reads inside one open SOQL scope

Database.Cursor (standard)

`fetch(offset, count)`
Forward by offset into scope (infinite scroll)

Database.PaginationCursor (GA)

`fetchPage(start, pageSize)`
Page-oriented access; CursorFetchResult



Not a SOQL cursor pattern:

List<SObject> = [SELECT ...]

all rows in heap at once
cursor: one SOQL scope, many fetches

Usecases for Apex Cursors

Interactive

- Analyst large data view
- Reporting and drilldown
- Preview job records to process

```
apex>
Database.PaginationCursor c =
  Database.getPaginationCursor(
    soql, AccessLevel.USER_MODE);
Database.CursorFetchResult r =
  c.fetchPage(start, pageSize);
```

Batch

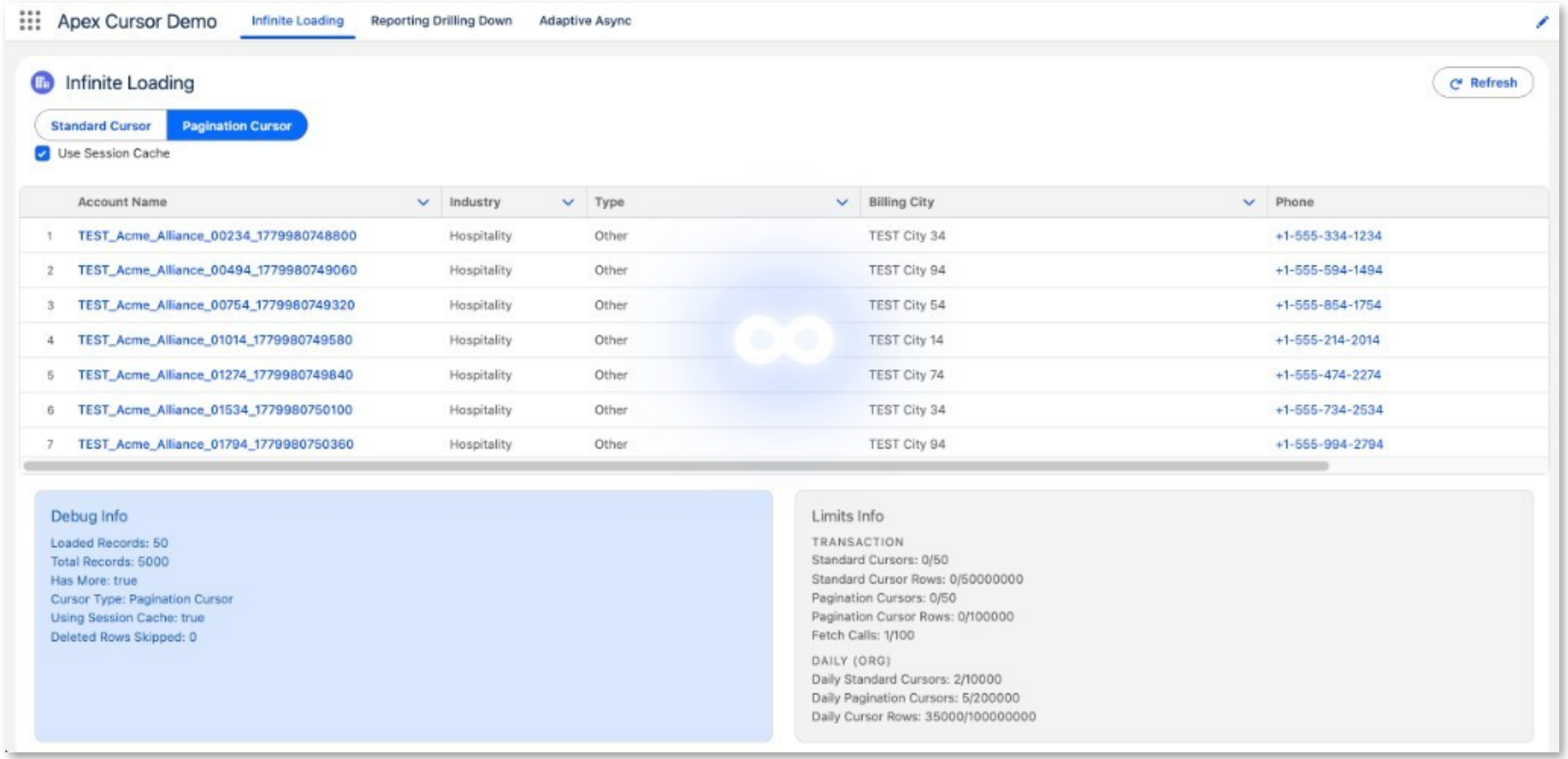
- Orchestrate process execution
- Look ahead to optimize resource usage
- Safer immutable record set

```
apex>
Database.Cursor c =
  Database.getCursor(
    soql, AccessLevel.USER_MODE);
List<Account> batch =
  c.fetch(offset, pageSize);
```

Architecture

24hr cached large record sets | Scalable replacement to SOQL OFFSET | Bidirectional navigation

Demo 1: Virtual Data Table



Demo 1: Highlights

1

```
apex>
Database.Cursor c =
  Database.getCursor(
    soql,
    AccessLevel.USER_MODE);
```

2

```
apex>
List<Account> batch =
  c.fetch(offset,
    batchSize);
result.hasMore =
  offset < c.getNumRecords();
```

3

```
apex>
const result = await
  loadMoreRecords({
    request, useSessionCache
  });
```

- Infinite scroll loads 50 rows at a time — no SOQL OFFSET
- Cursor held in LWC state or Cache.Session across Apex calls
- Live limit panel surfaces cursor, fetch, and row governor usage
- Switch between standard and pagination cursor modes in one LWC

Demo 2: Interactive Reporting and Drilldown

Demo 2: Highlights

1

```
apex>
Database.PaginationCursor pc =
  Database.getPaginationCursor(
    soql,
    AccessLevel.USER_MODE);
Cache.Session.put(
  sessionKey(product), pc);
```

2

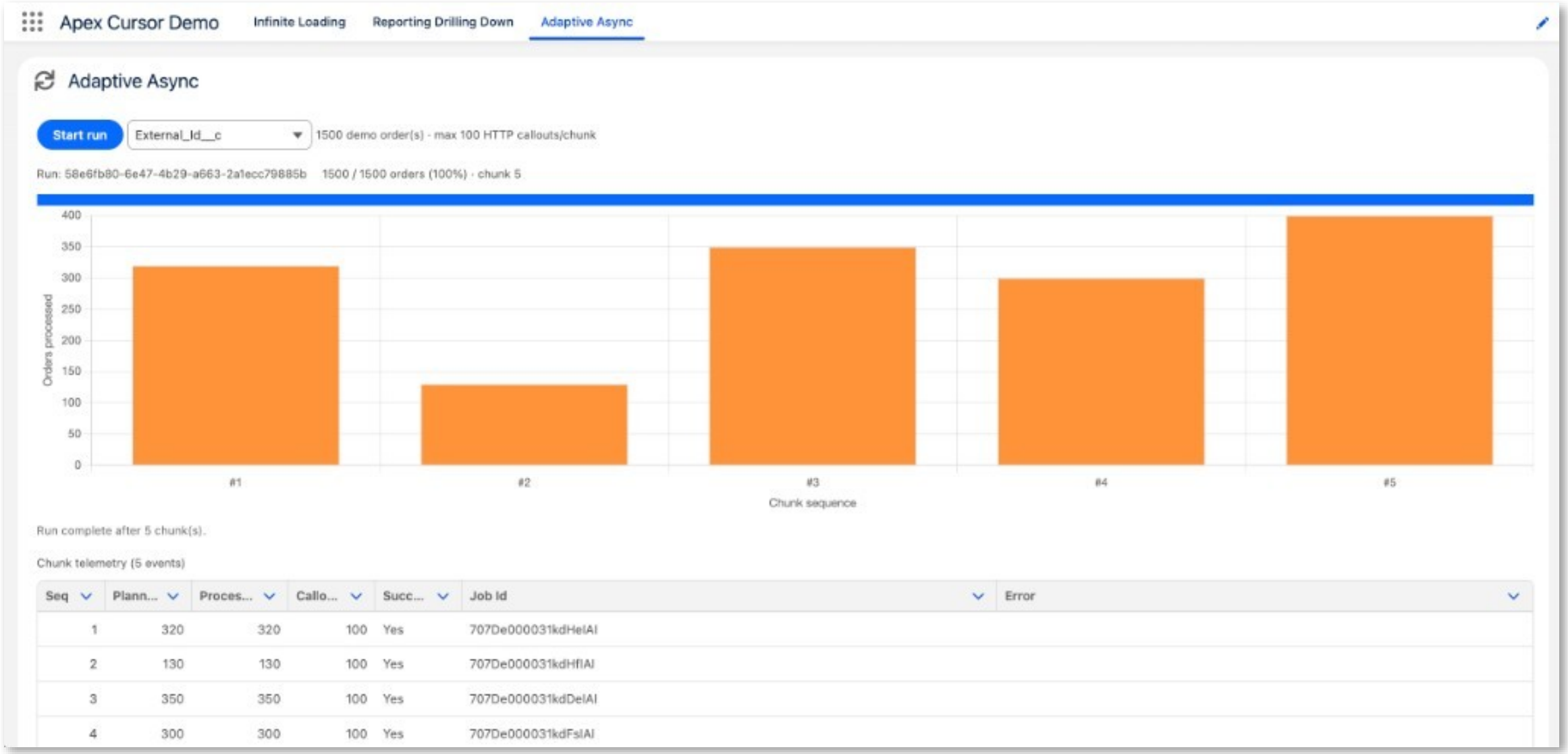
```
apex>
fetchResult = pc.fetchPage(
  dayIndex, 1);
deletedRows =
  fetchResult
    .getNumDeletedRecords();
```

3

```
apex>
const detail = await
  fetchDetailRange({
    product,
    startIndex, endIndex
  });
```

- PaginationCursor builds the monthly chart with one fetchPage per month
- Session cache keeps the cursor for brush-selection drill-down
- Index-based fetchPage maps chart range to daily observation rows
- fetchPage reports deleted rows when underlying data changes

Demo 3: Adaptive Async



Demo 3: Highlights

1

```
apex>
Database.Cursor cursor =
  Database.getCursor(
    soql,
    AccessLevel.USER_MODE);
System.enqueueJob(
  new AdaptiveOrderWorkerQueueable(
    runId, cursor, 1), options);
```

2

```
apex>
for (Order row :
  cursor.fetch(pos,
    batchSize)) {
  rowCallouts =
    rowCalloutsFor(row);
  // pack to callout budget
```

3

```
apex>
if (moreOrders) {
  sequence++;
  System.enqueueJob(this);
}
System.attachFinalizer(this);
```

- One cursor opened in Apex and passed through chained queueable jobs
- Chunk size adapts to remaining HTTP callout budget per transaction
- Finalizer publishes platform events after each chunk commits
- Cursor position travels on the job — no Database.Stateful required

Limits

Limit	Pagination	Standard
Cursors (per transaction)	50	50
Max rows per cursor	100,000	50,000,000
Fetch calls (per transaction)	100	100
Daily cursors (per org)	200,000	10,000
Daily cursor rows (per org)	1,000,000,000	1,000,000,000

- Max rows per cursor caps each cursor's result set size — not a per-transaction row governor.
- Fetch calls are shared: Cursor.fetch and PaginationCursor.fetchPage both count toward the 100-call limit.

Tips and Tricks

Practical patterns from the Apex Cursor demos

Instrument limits first

Show cursor, fetch, and row limits in the UI before users hit governor walls.

Session cache vs LWC state

Cache.Session survives Apex calls; LWC-held cursor avoids server-side state.

Pagination for drill-down

PaginationCursor + fetchPage for deleted rows and chart index ranges.

Stable ORDER BY

Sort on a unique key so offsets and page indexes stay predictable.

Right-size each fetch

Smaller batches feel snappier; larger batches mean fewer fetch calls.

USER_MODE by default

Pass AccessLevel.USER_MODE on getCursor unless elevated access is required.

LONDON'S CALLING

SmarterPay

PDF BUTLER flexpricer provar Gearset Nutrient SoClo INVENTORY CLOUD PERSPECTIVE

WITH secure avonni TITAN Elements .cloud Super mums Vinton From Conversations to Data

testsigma autorabit ClickLearn COPADO FormAssembly

Q&A

#LDNsCall #LC26

Andrew Fawcett

andy@andyinthecloud.com | LinkedIn

LONDON'S CALLING

SmarterPay

PDF BUTLER flexpricer provar Gearset Nutrient SoClo INVENTORY CLOUD PERSPECTIVE

WITH secure avonni TITAN Elements .cloud Super mums Vinton From Conversations to Data

testsigma autorabit ClickLearn COPADO FormAssembly

Thank You

#LDNsCall #LC26

Andrew Fawcett

andy@andyinthecloud.com | LinkedIn